

ASSIGNMENT 2

Software Floating-Point Unit for TOY-16

16-bits FP Formate

S	E E E E E	M M M M M M M M M M
1	5	10

Bias = 15, Hidden Bit=1

⚙️ 5-Step Algorithm

- 1 **EXTRACT** → Split Sign, Exp, Mantissa
- 2 **ALIGN** → Shift smaller mantissa right
- 3 **ADD** → Add mantissas
- 4 **NORMALIZE** → Fix overflow/underflow
- 5 **PACK** → Combine back to 16-bit

```
function fp16_add(a, b):  
    sa, ea, ma = extract(a)  
    sb, eb, mb = extract(b)  
    if ea > eb:  
        mb = shift_right(mb, ea - eb)  
    elif eb > ea:  
        ma = shift_right(ma, eb - ea)  
    exp = max(ea, eb)  
    mant = ma + mb  
    mant, exp = normalize(mant, exp)  
    return pack(sa ^ sb, exp, mant)
```

1st Number (2.12)

Result (6.37)

Extraction code

Packing

2nd Number (4.25)

Constants

Normalization

Arithmetic

Alignment

00: 0000
01: 403D
02: 4440
03: 465E

04: 000F
05: 7C00
06: 03FF
07: 0400

08: 000A
09: 0001
0A: FFFF
0B: 0800

0C: 0000
0D: 0000
0E: 0000
0F: 0000

10: 8101
11: 8202
12: 8304
13: 6413

14: 6523
15: 8605
16: 3716
17: 3826

18: 8908
19: 6779
1A: 6889
1B: 8A06

1C: 3B1A
1D: 3C2A
1E: 8D07
1F: 1BBD

20: 1CCD
21: 2178
22: C12D
23: D12B

24: 8E0A
25: 411E
26: 8209
27: 1112

28: 6BB1
29: 1771
2A: C02D
2B: 6CC1

2C: 1881
2D: 11BC
2E: 820B
2F: 3221

30: C234
31: 7301
32: 1773
33: 6113

34: 411D
35: 5779
36: 4117
37: 9103

38: 0000
39: 0000
3A: 0000
3B: 0000

3C: 0000
3D: 0000
3E: 0000
3F: 0000